

/tikz/,/tikz/graphs/
conversions/canvas coordinate/.code=1 , conversions/coordinate/.code=1

Jahangirnagar University

Department of Computer Science and Engineering

2nd Year 1st Semester B.Sc. (Hons.) Final

Examination – 2021

Course: CSE 209 – Algorithms-I*Complete Answer Script***SECTION-Contents**

Section-1 (CO1)	3
Section-2 (CO2)	6
Section-3 (CO3)	8
Section-4 (CO4)	11
Section-4 continued	13
Section-5 (CO5)	15

SECTION-1 SECTION-1 (CO1)

Seven questions; answer any six.

Question (a)

(i) Definition of Algorithm:

An **algorithm** is a finite, well-defined sequence of unambiguous instructions or steps that takes some input (possibly none), performs a computation, and produces an output in a finite amount of time to solve a problem or accomplish a task.

(ii) Differentiation between Big-O, Big-Omega and Big-Theta:

- **Big-O (O):** Asymptotic *upper bound*. $f(n) = O(g(n))$ means f grows no faster than g in the worst case.
- **Big-Omega (Ω):** Asymptotic *lower bound*. $f(n) = \Omega(g(n))$ means f grows at least as fast as g in the best case.
- **Big-Theta (Θ):** *Tight* (exact asymptotic) bound. $f(n) = \Theta(g(n))$ means f is bounded both above and below by g , i.e. $f = O(g)$ and $f = \Omega(g)$ simultaneously.

Question (b)

Worst-case and Best-case for Sorting Algorithms:

Algorithm	Best Case	Worst Case
Insertion Sort	$\Theta(n)$ — already sorted	$\Theta(n^2)$ — reverse sorted
Quick Sort	$\Theta(n \log n)$ — balanced partitions	$\Theta(n^2)$ — already sorted / all equal

Note: For Insertion Sort there is a distinct best case (array already sorted, only $n - 1$ comparisons). For Quick Sort with a naive pivot, the worst case occurs when the pivot is always the minimum or maximum element.

Question (c)

Properties of Dynamic Programming (DP):

1. **Optimal Substructure:** An optimal solution to the problem contains within it optimal solutions to sub-problems.
2. **Overlapping Sub-problems:** The same sub-problems are solved multiple times; DP stores their results (*memoisation* or *tabulation*) to avoid redundant computation.
3. **Bottom-up or Top-down Construction:** Solutions are built from the smallest sub-problems upward (bottom-up table) or computed recursively with caching (top-down).
4. **Polynomial Time:** DP typically converts exponential brute-force into polynomial-time solutions by reusing stored results.

Question (d)**Limitations of Binary Search:**

- The array/list must be *sorted* beforehand.
- Requires *random access* (does not work well on linked lists).
- Not suitable for unsorted or frequently changing data.

Binary Search on $\{7, 3, 0, -1, -2\}$ for Key = 1:

Binary Search requires a sorted array. Sort: $\{-2, -1, 0, 3, 7\}$.

$$\text{low} = 0, \text{high} = 4, \text{mid} = 2 \Rightarrow A[2] = 0 < 1 \Rightarrow \text{low} = 3$$

$$\text{low} = 3, \text{high} = 4, \text{mid} = 3 \Rightarrow A[3] = 3 > 1 \Rightarrow \text{high} = 2$$

$$\text{low} = 3 > \text{high} = 2 \Rightarrow \textbf{Key not found}$$

Binary Search *is* possible on the sorted version; the key 1 is simply not present in the sequence.

Question (e)**Divide and Conquer Paradigm:**

Divide and Conquer is an algorithm design strategy that works in three stages:

1. **Divide:** Split the problem into smaller sub-problems of the same type.
2. **Conquer:** Solve each sub-problem recursively (base cases are solved directly).
3. **Combine:** Merge/combine the solutions of sub-problems to obtain the solution to the original problem.

Examples: Merge Sort, Quick Sort, Binary Search, Strassen's Matrix Multiplication.

Question (f)**Weakly vs. Strongly Connected Component:**

- **Weakly Connected Component (WCC):** In a directed graph, a set of vertices is weakly connected if the underlying *undirected* version (ignoring edge directions) is connected. There may be no directed path between some pairs.
- **Strongly Connected Component (SCC):** A maximal set of vertices such that there is a *directed path* from every vertex to every other vertex in the set. Every vertex can reach every other vertex following directed edges.

Question (g)**Graph Representation Techniques:**

1. **Adjacency Matrix:** A $V \times V$ Boolean (or weighted) matrix where entry $(i, j) = 1$ if edge (i, j) exists.
 - *Advantage:* $O(1)$ edge lookup; simple implementation.
 - *Disadvantage:* $O(V^2)$ space; inefficient for sparse graphs.
2. **Adjacency List:** Each vertex stores a list of its neighbours.
 - *Advantage:* $O(V + E)$ space; efficient for sparse graphs and traversal.
 - *Disadvantage:* $O(\deg(v))$ edge lookup (not $O(1)$).

When to prefer: Use the adjacency matrix for dense graphs or when fast edge-existence queries are needed; use the adjacency list for sparse graphs or algorithms that iterate over neighbours (BFS, DFS, Dijkstra, etc.).

SECTION-2 SECTION-2 (CO2)

Four questions; answer any three.

Question 2(a) — 2-way and 3-way Merge Sort

Given array: 38, 27, 43, 3, 9, 82, 10, 12, 20, 18

2-way Merge Sort (standard): Split into two halves recursively, then merge.

Level 0: [38, 27, 43, 3, 9, 82, 10, 12, 20, 18]
 Level 1: [38, 27, 43, 3, 9] [82, 10, 12, 20, 18]
 Level 2: [38, 27] [43, 3, 9] [82, 10] [12, 20, 18]
 Level 3: [38] [27] [43] [3, 9] [82] [10] [12] [20, 18]
 Level 4: [43] [3] [9] [20] [18]
 Merge up:
 [27, 38] [3, 9, 43] [10, 82] [12, 18, 20]
 [3, 9, 27, 38, 43] [10, 12, 18, 20, 82]
 Final: [3, 9, 10, 12, 18, 20, 27, 38, 43, 82]

3-way Merge Sort: Split into three parts recursively, then 3-way merge.

Level 0: [38, 27, 43, 3, 9, 82, 10, 12, 20, 18]
 Level 1: [38, 27, 43] [3, 9, 82] [10, 12, 20, 18]
 Level 2: [38] [27, 43] [3] [9, 82] [10, 12] [20, 18]
 Merge: [27, 38, 43] [3, 9, 82] [10, 12, 18, 20]
 Final 3-way merge: [3, 9, 10, 12, 18, 20, 27, 38, 43, 82]

Time Complexity Comparison:

	2-way Merge Sort	3-way Merge Sort
Best / Average / Worst	$\Theta(n \log_2 n)$	$\Theta(n \log_3 n)$
Relative factor	$\log_2 n$	$\log_3 n = \frac{\log_2 n}{\log_2 3} \approx 0.63 \log_2 n$

Both are $O(n \log n)$; 3-way has fewer recursive levels but more work per merge step. In practice the improvement is modest and both are $O(n \log n)$.

Question 2(b) — Quick Sort on {4, 6, 2, 8, 1, 5, 3}

Using the *last element* as pivot each time:

Step 1: Array [4, 6, 2, 8, 1, 5, 3], pivot = 3

After partition: [2, 1, 3, 8, 6, 5, 4] (pivot at index 2)

Step 2a: Left [2, 1], pivot = 1

$\Rightarrow [1, 2]$

Step 2b: Right [8, 6, 5, 4], pivot = 4

$\Rightarrow [4, 6, 5, 8] \Rightarrow$ pivot at index 0

Right of 4: [6, 5, 8], pivot = 8: already [6, 5, 8] $\Rightarrow$ [5, 6, 8] after sub-sort.

Final sorted array: [1, 2, 3, 4, 5, 6, 8]

Time Complexity:

- Best/Average: $O(n \log n)$ — balanced partitions
- Worst: $O(n^2)$ — already sorted or reverse sorted with naive pivot

Question 2(c) — Bellman-Ford Algorithm

Graph (from figure): Vertices $\{1, 2, 3, 4\}$, Edges (directed, with weights): $1 \rightarrow 2$ (5), $1 \rightarrow 3$ (5), $3 \rightarrow 2$ (7), $2 \rightarrow 4$ (15), $3 \rightarrow 4$ (-15(?))

Reading the graph image carefully: edges are $1 \rightarrow 3$ (5), $1 \rightarrow 2$ (4), $3 \rightarrow 2$ (7), $2 \rightarrow 4$ (15), $3 \rightarrow 4$ (-15) (approximate weights from figure).

Bellman-Ford from source vertex 1:

Initialise: $d[1] = 0$, $d[2] = d[3] = d[4] = \infty$

Iteration 1 (relax all edges):

$$\begin{aligned}
 1 \rightarrow 3 \text{ (5)} : \quad d[3] &= \min(\infty, 0 + 5) = 5 \\
 1 \rightarrow 2 \text{ (4)} : \quad d[2] &= \min(\infty, 0 + 4) = 4 \\
 3 \rightarrow 2 \text{ (7)} : \quad d[2] &= \min(4, 5 + 7) = 4 \\
 2 \rightarrow 4 \text{ (15)} : \quad d[4] &= \min(\infty, 4 + 15) = 19 \\
 3 \rightarrow 4 \text{ (-15 approx)} : \quad d[4] &= \min(19, 5 + (-15)) = -10
 \end{aligned}$$

Iteration 2: No further updates (distances stabilise).

Iteration 3 ($V - 1 = 3$): No change.

Negative Cycle Check: Perform one more relaxation — no update, so *no negative cycle*.

Shortest paths from vertex 1: $d[1] = 0$, $d[2] = 4$, $d[3] = 5$, $d[4] = -10$

Conclusion: Yes, it is possible to find the shortest path with Bellman-Ford. The algorithm handles negative edge weights (as long as there are no negative cycles).

Complexity: $O(VE)$

Question 2(d) — Topological Sort

Given DAG (from figure, nodes appear to be 7, 5, 1, 2, 3, 4, 6 with directed edges forming a DAG):

Using **DFS-based topological sort** (Kahn's algorithm or DFS finish-time ordering):

1. Compute in-degrees of all vertices.
2. Enqueue all vertices with in-degree 0.
3. Repeatedly dequeue a vertex u , add to topological order, decrement in-degree of all neighbours; enqueue those that reach 0.

Topological Order (based on the DAG structure in the image, reading left to right):

$$7 \rightarrow 5 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6$$

(Exact order depends on edge directions in the original figure.)

Complexity: $O(V + E)$

SECTION-3 SECTION-3 (CO3)

Four questions; answer any three.

Question 3(a) — Optimal Job Scheduling (Minimize Maximum Lateness)

Given Table:

	1	2	3	4	5	6	7
t_j (processing time)	1	2	3	4	3	2	3
d_j (deadline)	5	4	3	2	7	4	2
D (given row)	6	8	9	9	14	15	17

Strategy: Earliest Deadline First (EDF) — sort jobs by deadline d_j in non-decreasing order. This is provably optimal for minimising maximum lateness.

Sorted order by deadline:

Job	t_j	d_j	Start/Finish time
7	3	2	[0, 3)
4	4	2	[3, 7)
3	3	3	[7, 10)
2	2	4	[10, 12)
6	2	4	[12, 14)
1	1	5	[14, 15)
5	3	7	[15, 18)

Lateness $L_j = \max(0, C_j - d_j)$:

- Job 7: $C = 3, d = 2, L = 1$
- Job 4: $C = 7, d = 2, L = 5$
- Job 3: $C = 10, d = 3, L = 7$
- Job 2: $C = 12, d = 4, L = 8$
- Job 6: $C = 14, d = 4, L = 10$
- Job 1: $C = 15, d = 5, L = 10$
- Job 5: $C = 18, d = 7, L = 11$

$L_{\max} = 11$

Complexity: Sorting takes $O(n \log n)$; scheduling is $O(n)$. **Overall:** $O(n \log n)$.

Question 3(b) — Heap Sort on {5, 6, 3, 1, 4, 2}**Using Max-Heap.****Step 1 — Build Max-Heap** (heapify from last non-leaf):

Initial: [5, 6, 3, 1, 4, 2]
 Heapify(1): parent=6, children=1,4 -> no swap needed at i=1
 Heapify(0): parent=5, children=6,3 -> swap 5 and 6
 [6, 5, 3, 1, 4, 2]
 check i=1: parent=5, children=1,4 -> no swap
 Max-Heap: [6, 5, 3, 1, 4, 2]

Step 2 — Sort (swap root with last, reduce heap size, heapify root):

Swap A[0] with A[5]: [2,5,3,1,4,6], heap=[2,5,3,1,4]
 Heapify(0): 2<5 -> swap -> [5,2,3,1,4] -> 2<4 -> swap -> [5,4,3,1,2]
 Swap A[0] with A[4]: [2,4,3,1,5,6], heap=[2,4,3,1]
 Heapify(0): 2<4 -> swap -> [4,2,3,1] -> no more
 Swap A[0] with A[3]: [1,2,3,4,5,6], heap=[1,2,3]
 Heapify(0): 1<3 -> swap -> [3,2,1]
 Swap A[0] with A[2]: [1,2,3,4,5,6], heap=[1,2]
 Heapify(0): 1<2 -> swap -> [2,1]
 Swap A[0] with A[1]: [1,2,3,4,5,6], heap=[1]
 Done.

Sorted: [1, 2, 3, 4, 5, 6]**Complexity:**

- Build heap: $O(n)$
- n extractions each needing $O(\log n)$ heapify: $O(n \log n)$
- **Overall:** $O(n \log n)$ in all cases.

Question 3(c) — Longest Common Subsequence (LCS) $X = \text{"BACABACB"}, Y = \text{"ACBAABAC"}$ **Recursive formulation:**

$$\text{LCS}(i, j) = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ 1 + \text{LCS}(i-1, j-1) & \text{if } X[i] = Y[j] \\ \max(\text{LCS}(i-1, j), \text{LCS}(i, j-1)) & \text{otherwise} \end{cases}$$

DP Table ($m = 8, n = 8$), rows = X , columns = Y :

	ϵ	A	C	B	A	A	B	A	C
ϵ	0	0	0	0	0	0	0	0	0
B	0	0	0	1	1	1	1	1	1
A	0	1	1	1	2	2	2	2	2
C	0	1	2	2	2	2	2	2	3
A	0	1	2	2	3	3	3	3	3
B	0	1	2	3	3	3	4	4	4
A	0	1	2	3	4	4	4	5	5
C	0	1	2	3	4	4	4	5	6
B	0	1	2	3	4	4	5	5	6

LCS Length = 6

One possible LCS: **ACABAC** (length 6).

Complexity: $O(mn)$ time and space, where $m = |X|, n = |Y|$.

Question 3(d) — Chinese Remainder Theorem (CRT)

Given arrays:

$$\text{num}[] = [3, 4, 5, 2, 3], \quad \text{rem}[] = [2, 3, 1, 2, 1]$$

(Reading from table: Num row = 3, 4, 5, 2, 3; rem row = 2, 3, 1, 2, 1... adjusting to the visible values.)

We need the smallest positive x such that:

$$x \bmod \text{num}[i] = \text{rem}[i] \quad \forall i$$

Using the Inverse-modulo based CRT implementation:

1. Compute $N = \prod \text{num}[i]$.
2. For each i : $N_i = N/\text{num}[i]$.
3. Find $M_i = N_i^{-1} \pmod{\text{num}[i]}$ (modular inverse using Extended Euclidean).
4. $x = (\sum_i \text{rem}[i] \cdot N_i \cdot M_i) \bmod N$.

Example with num = [3, 4, 5], rem = [2, 3, 1]:

$$N = 60, \quad N_0 = 20, \quad N_1 = 15, \quad N_2 = 12$$

$$M_0 = 20^{-1} \bmod 3 = 2^{-1} \bmod 3 = 2$$

$$M_1 = 15^{-1} \bmod 4 = 3^{-1} \bmod 4 = 3$$

$$M_2 = 12^{-1} \bmod 5 = 2^{-1} \bmod 5 = 3$$

$$x = (2 \cdot 20 \cdot 2 + 3 \cdot 15 \cdot 3 + 1 \cdot 12 \cdot 3) \bmod 60 = (80 + 135 + 36) \bmod 60 = 251 \bmod 60 = 11$$

Verify: $11 \bmod 3 = 2 \checkmark$, $11 \bmod 4 = 3 \checkmark$, $11 \bmod 5 = 1 \checkmark$.

Complexity: $O(k \log(\min(\text{num}[i])))$ where k is the number of congruences.

SECTION-4 SECTION-4 (CO4)

Four questions; answer any three.

Question 4(a) — Huffman Coding

Application: Huffman coding is used in **lossless data compression**. It assigns shorter binary codes to more frequent characters and longer codes to less frequent ones, minimising the total encoded length. It is widely used in file compression formats (ZIP, GZIP), image formats (JPEG, PNG), and communication protocols.

Given frequencies:

Character	a	b	c	d	E	f
Frequency	34	14	12	16	14	10

Huffman Tree Construction (Min-Heap / Priority Queue):

1. Initial nodes (sorted): $f(10), c(12), b(14), E(14), d(16), a(34)$
2. Merge $f(10) + c(12) = 22$: nodes: $b(14), E(14), d(16), fc(22), a(34)$
3. Merge $b(14) + E(14) = 28$: nodes: $d(16), fc(22), bE(28), a(34)$
4. Merge $d(16) + fc(22) = 38$: nodes: $bE(28), a(34), dfc(38)$
5. Merge $bE(28) + a(34) = 62$: nodes: $dfc(38), bEa(62)$
6. Merge $dfc(38) + bEa(62) = 100$: Root

Codewords (left = 0, right = 1):

Character	Frequency	Codeword
a	34	11
b	14	100
E	14	101
d	16	00
c	12	011
f	10	010

Average codeword length:

$$L = \frac{34 \cdot 2 + 14 \cdot 3 + 14 \cdot 3 + 16 \cdot 2 + 12 \cdot 3 + 10 \cdot 3}{34 + 14 + 14 + 16 + 12 + 10} = \frac{68 + 42 + 42 + 32 + 36 + 30}{100} = \frac{250}{100} = 2.5 \text{ bits/symbol}$$

Question 4(b) — All-Pairs Shortest Path (Floyd-Warshall)**(i) Why Dijkstra cannot be used:**

The graph contains **negative edge weights** (e.g., -4 , -5 visible in the figure). Dijkstra's algorithm assumes all edge weights are non-negative; with negative weights it may produce incorrect shortest paths. Hence, an algorithm that handles negative weights must be used.

(ii) Algorithm: Floyd-Warshall for all-pairs shortest paths.

Vertices: $\{1, 2, 3, 4, 5\}$ (from figure). Let the weight matrix W be initialised from the graph (0 on diagonal, ∞ for non-edges, edge weight otherwise).

Floyd-Warshall Recurrence:

$$d^{(k)}[i][j] = \min(d^{(k-1)}[i][j], d^{(k-1)}[i][k] + d^{(k-1)}[k][j])$$

for $k = 1, 2, \dots, V$.

Reading edge weights from the figure: Edges (approximate): $1 \rightarrow 2$ (2), $1 \rightarrow 5$ (-4 ?), $2 \rightarrow 3$ (3), $2 \rightarrow 4$ (4), $3 \rightarrow 4$ (8), $3 \rightarrow 2$ (3)(?), $5 \rightarrow 4$ (6), $5 \rightarrow 1$ (7), $4 \rightarrow 3$ (-5)(?), $1 \rightarrow 3$ (3)(?), and more.

Initial distance matrix D (using visible weights; ∞ = no direct edge):

$$D = \begin{pmatrix} 0 & 2 & 3 & \infty & -4 \\ \infty & 0 & 3 & 4 & \infty \\ \infty & \infty & 0 & 8 & \infty \\ \infty & \infty & -5 & 0 & \infty \\ \infty & 7 & \infty & 6 & 0 \end{pmatrix}$$

Applying Floyd-Warshall (3 intermediate vertex updates shown):

After $k = 5$ iterations, the final shortest path matrix is:

$$D^* = \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix}$$

Complexity of Floyd-Warshall: $O(V^3)$

SECTION-5 SECTION-4 (CO4) continued**Question — 0/1 Knapsack Problem, $K_c = 6$** **Items:**

Item	Value	Weight
1	12	2
2	10	4
3	9	3
4	6	1

DP Table $dp[i][w]$ = maximum value using first i items with capacity w :

$i \backslash w$	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1 (v=12,w=2)	0	0	12	12	12	12	12
2 (v=10,w=4)	0	0	12	12	12	12	22
3 (v=9,w=3)	0	0	12	12	12	21	22
4 (v=6,w=1)	0	6	12	18	18	21	27

Optimal Value = 27**Items selected** (back-track): Item 4 (w=1, v=6) + Item 1 (w=2, v=12) + Item 3 (w=3, v=9) = weight $1 + 2 + 3 = 6 \leq 6$, value = 27.**Complexity:** $O(nW)$ where $n = 4, W = 6$. This is **pseudo-polynomial**.**Question — Activity Selection Problem****Given Activities:**

Activity	Start Time s_i	Finish Time f_i
1	1	3
2	0	4
3	1	2
4	4	6
5	2	9
6	5	8
7	3	5
8	4	5

Greedy Strategy: Sort by finish time; greedily select next compatible activity.**Sorted by finish time:** 3(1,2), 1(1,3), 2(0,4), 7(3,5), 8(4,5), 4(4,6), 6(5,8), 5(2,9)**Selection:**

1. Select Activity 3 ($s = 1, f = 2$); last finish = 2.
2. Activity 1: $s = 1 < 2$ — skip.
3. Activity 2: $s = 0 < 2$ — skip.

4. Activity 7: $s = 3 \geq 2$ — **select**; last finish = 5.
5. Activity 8: $s = 4 < 5$ — skip.
6. Activity 4: $s = 4 < 5$ — skip.
7. Activity 6: $s = 5 \geq 5$ — **select**; last finish = 8.
8. Activity 5: $s = 2 < 8$ — skip.

Maximum non-overlapping set: $\{3, 7, 6\}$ — **3 activities.**

Complexity: $O(n \log n)$ for sorting + $O(n)$ for selection = $O(n \log n)$.

SECTION-6 SECTION-5 (CO5)

Three questions; answer any two.

Question 5(a) — DFS Edge Classification & SCC

Graph vertices: $A, B, C, D, E, F, G, H, I$

Reading the graph: edges appear to be: $A \rightarrow I, A \rightarrow D, D \rightarrow E, D \rightarrow G, D \rightarrow H, E \rightarrow H, I \rightarrow E, B \rightarrow C, B \rightarrow F, C \rightarrow F, F \rightarrow G$

DFS from A (visit in order encountered):

DFS traversal order (approximate): $A \rightarrow I \rightarrow E \rightarrow H \rightarrow D \rightarrow G \rightarrow \dots$

Edge types in DFS:

- **Tree Edge:** Edge (u, v) where v is first discovered via u . These form the DFS tree.
Examples: $A \rightarrow I, I \rightarrow E, E \rightarrow H, A \rightarrow D, D \rightarrow G$
- **Back Edge:** Edge (u, v) where v is an ancestor of u in the DFS tree (creates a cycle in directed graph — indicates a cycle / SCC).
- **Forward Edge:** Edge (u, v) where v is a descendant of u but not a direct tree edge.
Example: $A \rightarrow D$ (if D already discovered as descendant)
- **Cross Edge:** Edge (u, v) where v is neither ancestor nor descendant.
Example: $D \rightarrow H$ if H is already fully processed.

Strongly Connected Components (SCC) using Kosaraju's Algorithm:

Step 1: Perform DFS on original graph G ; record finish times.

Step 2: Transpose the graph G^T (reverse all edges).

Step 3: Perform DFS on G^T in decreasing order of finish times; each DFS tree is an SCC.

SCCs (based on graph structure): Since the graph appears to have mostly one-directional paths with a few back edges, the SCCs are likely mostly individual vertices or small groups. Based on the image:

$SCC_1 = \{A\}, SCC_2 = \{I\}, SCC_3 = \{B, C, F\}, SCC_4 = \{D\}, SCC_5 = \{E, H\}, SCC_6 = \{G\}$

(Exact SCCs depend on edge directions in the original figure.)

Complexity of SCC (Kosaraju's): $O(V + E)$

Question 5(b) — Subset Sum using Dynamic Programming

Set: $S = \{3, 34, 4, 12, 5, 2\}$, **Target sum** = 9.

DP Table: $dp[i][j]$ = True if there is a subset of the first i elements with sum j .

Recurrence:

$$dp[i][j] = dp[i-1][j] \vee (j \geq S[i] \wedge dp[i-1][j-S[i]])$$

Table (T = True, F = False):

	0	1	2	3	4	5	6	7	8	9
$\{\}$	T	F	F	F	F	F	F	F	F	F
$\{3\}$	T	F	F	T	F	F	F	F	F	F
$\{3, 34\}$	T	F	F	T	F	F	F	F	F	F
$\{3, 34, 4\}$	T	F	F	T	T	F	F	T	F	F
$\{3, 34, 4, 12\}$	T	F	F	T	T	F	F	T	F	F
$\{3, 34, 4, 12, 5\}$	T	F	F	T	T	T	F	T	T	T
$\{3, 34, 4, 12, 5, 2\}$	T	F	T	T	T	T	T	T	T	T

$dp[6][9] = \text{True}$, so **yes, a subset exists with sum 9.**

Subset found: $\{4, 5\} \Rightarrow 4 + 5 = 9 \checkmark$ (also $\{3, 4, 2\} = 9$, $\{2, 5, 2\}$, etc.)

Recursion:

$\text{SubsetSum}(S, n, \text{target}) = \text{SubsetSum}(S, n-1, \text{target}) \vee \text{SubsetSum}(S, n-1, \text{target} - S[n])$

Complexity: $O(n \cdot W)$ where $n = |S| = 6$, $W = 9$ (target sum).

Question 5(c) — Recurrence $f_n = (2f_{n-1} + 3f_{n-2}) \bmod 101$

(i) **Solution for any value of n :**

The recurrence is linear: $f_n = (2f_{n-1} + 3f_{n-2}) \bmod 101$.

This can be expressed in matrix form:

$$\begin{pmatrix} f_n \\ f_{n-1} \end{pmatrix} = \begin{pmatrix} 2 & 3 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} f_{n-1} \\ f_{n-2} \end{pmatrix} \pmod{101}$$

Let $M = \begin{pmatrix} 2 & 3 \\ 1 & 0 \end{pmatrix}$. Then:

$$\begin{pmatrix} f_n \\ f_{n-1} \end{pmatrix} = M^{n-1} \begin{pmatrix} f_1 \\ f_0 \end{pmatrix} \pmod{101}$$

Assuming initial conditions $f_0 = 0$, $f_1 = 1$ (typical), for any n compute $M^{n-1} \pmod{101}$ using **fast matrix exponentiation**.

Brief description: Matrix exponentiation computes M^k in $O(\log k)$ matrix multiplications by the repeated squaring:

$$M^k = \begin{cases} I & k = 0 \\ (M^{k/2})^2 & k \text{ even} \\ M \cdot M^{k-1} & k \text{ odd} \end{cases}$$

(ii) **Compute f_{40} using matrix exponentiation:**

We compute $M^{39} \pmod{101}$.

Assuming $f_0 = 0$, $f_1 = 1$:

Compute successive values mod 101:

$$\begin{aligned}f_0 &= 0 \\f_1 &= 1 \\f_2 &= (2 \cdot 1 + 3 \cdot 0) \bmod 101 = 2 \\f_3 &= (2 \cdot 2 + 3 \cdot 1) \bmod 101 = 7 \\f_4 &= (2 \cdot 7 + 3 \cdot 2) \bmod 101 = 20 \\f_5 &= (2 \cdot 20 + 3 \cdot 7) \bmod 101 = 61 \\f_6 &= (2 \cdot 61 + 3 \cdot 20) \bmod 101 = 182 \bmod 101 = 81 \\f_7 &= (2 \cdot 81 + 3 \cdot 61) \bmod 101 = 345 \bmod 101 = 42 \\f_8 &= (2 \cdot 42 + 3 \cdot 81) \bmod 101 = 327 \bmod 101 = 24 \\f_9 &= (2 \cdot 24 + 3 \cdot 42) \bmod 101 = 174 \bmod 101 = 73 \\f_{10} &= (2 \cdot 73 + 3 \cdot 24) \bmod 101 = 218 \bmod 101 = 16 \\f_{11} &= (2 \cdot 16 + 3 \cdot 73) \bmod 101 = 251 \bmod 101 = 49 \\f_{12} &= (2 \cdot 49 + 3 \cdot 16) \bmod 101 = 146 \bmod 101 = 45 \\f_{13} &= (2 \cdot 45 + 3 \cdot 49) \bmod 101 = 237 \bmod 101 = 35 \\f_{14} &= (2 \cdot 35 + 3 \cdot 45) \bmod 101 = 205 \bmod 101 = 3 \\f_{15} &= (2 \cdot 3 + 3 \cdot 35) \bmod 101 = 111 \bmod 101 = 10 \\f_{16} &= (2 \cdot 10 + 3 \cdot 3) \bmod 101 = 29 \\f_{17} &= (2 \cdot 29 + 3 \cdot 10) \bmod 101 = 88 \\f_{18} &= (2 \cdot 88 + 3 \cdot 29) \bmod 101 = 263 \bmod 101 = 61 \\f_{19} &= (2 \cdot 61 + 3 \cdot 88) \bmod 101 = 386 \bmod 101 = 82 \\f_{20} &= (2 \cdot 82 + 3 \cdot 61) \bmod 101 = 347 \bmod 101 = 44 \\f_{21} &= (2 \cdot 44 + 3 \cdot 82) \bmod 101 = 334 \bmod 101 = 31 \\f_{22} &= (2 \cdot 31 + 3 \cdot 44) \bmod 101 = 194 \bmod 101 = 93 \\f_{23} &= (2 \cdot 93 + 3 \cdot 31) \bmod 101 = 279 \bmod 101 = 77 \\f_{24} &= (2 \cdot 77 + 3 \cdot 93) \bmod 101 = 433 \bmod 101 = 29 \\f_{25} &= (2 \cdot 29 + 3 \cdot 77) \bmod 101 = 289 \bmod 101 = 87 \\f_{26} &= (2 \cdot 87 + 3 \cdot 29) \bmod 101 = 261 \bmod 101 = 59 \\f_{27} &= (2 \cdot 59 + 3 \cdot 87) \bmod 101 = 379 \bmod 101 = 76 \\f_{28} &= (2 \cdot 76 + 3 \cdot 59) \bmod 101 = 329 \bmod 101 = 26 \\f_{29} &= (2 \cdot 26 + 3 \cdot 76) \bmod 101 = 280 \bmod 101 = 78 \\f_{30} &= (2 \cdot 78 + 3 \cdot 26) \bmod 101 = 234 \bmod 101 = 32 \\f_{31} &= (2 \cdot 32 + 3 \cdot 78) \bmod 101 = 298 \bmod 101 = 96 \\f_{32} &= (2 \cdot 96 + 3 \cdot 32) \bmod 101 = 288 \bmod 101 = 86 \\f_{33} &= (2 \cdot 86 + 3 \cdot 96) \bmod 101 = 460 \bmod 101 = 55 \\f_{34} &= (2 \cdot 55 + 3 \cdot 86) \bmod 101 = 368 \bmod 101 = 65 \\f_{35} &= (2 \cdot 65 + 3 \cdot 55) \bmod 101 = 295 \bmod 101 = 93 \\f_{36} &= (2 \cdot 93 + 3 \cdot 65) \bmod 101 = 381 \bmod 101 = 79 \\f_{37} &= (2 \cdot 79 + 3 \cdot 93) \bmod 101 = 437 \bmod 101 = 33 \\f_{38} &= (2 \cdot 33 + 3 \cdot 79) \bmod 101 = 303 \bmod 101 = 0 \\f_{39} &= (2 \cdot 0 + 3 \cdot 33) \bmod 101 = 99 \\f_{40} &= (2 \cdot 99 + 3 \cdot 0) \bmod 101 = 198 \bmod 101 = \boxed{97}\end{aligned}$$

$$f_{40} = 97$$

Complexity of Matrix Exponentiation: Each matrix multiplication is $O(k^2)$ where $k = 2$ (2×2 matrix), and we do $O(\log n)$ multiplications. Total: $O(k^2 \log n) = O(\log n)$ for fixed k .